Use Case Specification — CredVault

System Overview

Field	Details
System	CredVault Credit Card Management Platform
Version	1.0
Date	April 2026

1. Purpose

This document defines the functional behavior of the **CredVault** platform using structured use cases.

Each use case describes:

- Interactions between actors and the system
- Expected outcomes
- Handling of exceptional conditions

2. Actors

Actor	Type	Description
End User	Primary	Registered customer who manages cards, views bills, makes payments, and manages wallet
Admin	Primary	Platform administrator with elevated privileges for user, card, billing, and reward management
Google OAuth	Secondary	External identity provider for SSO login

Actor	Type	Description
Razorpay	Secondary	External payment gateway for wallet top-ups
Gmail SMTP	Secondary	External email service for OTP and notification delivery
System	Secondary	Automated processes (bill generation, payment expiration, saga orchestration)

3. Use Case Catalog

ID	Name	Primary Actor	Priority
UC-01	Register & Verify Account	End User	High
UC-02	Login (Email/Password)	End User	High
UC-03	Login via Google SSO	End User	Medium
UC-04	Manage Profile	End User	Medium
UC-05	Add Credit Card	End User	High
UC-06	Manage Cards	End User	Medium
UC-07	View Bills	End User	High
UC-08	View Statements	End User	Medium
UC-09	Pay Bill with OTP	End User	High
UC-10	Redeem Rewards	End User	Medium
UC-11	Top-Up Wallet	End User	Medium
UC-12	Reset Password	End User	Medium
UC-13	Manage Users	Admin	High
UC-14	Manage Reward Tiers	Admin	Low
UC-15	View Audit Logs	Admin	Low

4. Detailed Use Cases

UC-01: Register & Verify Account

Description: A new user creates an account and verifies email ownership via OTP.

Primary Actor: End User

Preconditions:

- User has a valid email address
- Email is not already registered

Postconditions:

- User account created and activated
- JWT token issued
- Welcome email sent

Primary Flow:

1. User navigates to registration page
2. User enters full name, email, and password
3. System validates input (email format, password strength, uniqueness)
4. System creates user with status `PendingVerification`
5. System generates 6-digit OTP (10-minute expiry)
6. System publishes `IUserRegistered` and `IUserOtpGenerated` events
7. Notification Service sends welcome email with OTP
8. System returns success response to user
9. User enters OTP from email
10. System validates OTP (match + not expired)
11. System marks user as `Active`
12. System generates and returns JWT token

Alternative Flows:

- **A1 — Email already exists:** System returns validation error at step 3
- **A2 — OTP expired:** System returns error at step 10; user requests new OTP via UC-01 resend
- **A3 — OTP mismatch:** System returns error at step 10; user retries (max 3 attempts)

Business Rules:

- Password minimum: 8 characters, 1 uppercase, 1 lowercase, 1 digit
- OTP is single-use and expires after 10 minutes
- Maximum 3 OTP verification attempts before requiring resend

UC-02: Login (Email/Password)

Description: A registered user authenticates with email and password to receive a JWT.

Primary Actor: End User

Preconditions:

- User account exists and is verified (`Active` status)

Postconditions:

- JWT token issued (60-minute validity)
- User redirected to dashboard

Primary Flow:

1. User navigates to login page
2. User enters email and password
3. System validates input format
4. System finds user by email
5. System validates password hash (BCrypt)
6. System checks user status is `Active`
7. System generates JWT token
8. System returns JWT + user profile

Alternative Flows:

- **A1 — Invalid credentials:** System returns generic error at step 5 (prevents user enumeration)
- **A2 — Account not verified:** System returns error at step 6 with prompt to verify email
- **A3 — Account suspended:** System returns error at step 6 indicating suspension

Business Rules:

- Error messages for invalid credentials are intentionally generic
- 5 failed login attempts within 15 minutes triggers temporary lockout
- JWT expires after 60 minutes; client auto-logs out on 401

UC-03: Login via Google SSO

Description: A user authenticates using their Google account without a platform password.

Primary Actor: End User

Secondary Actor: Google OAuth

Preconditions:

- User has a Google account

- Google Client ID configured in Identity Service

Postconditions:

- User account created (if new) or logged in (if existing)
- JWT token issued

Primary Flow:

1. User clicks "Sign in with Google"
2. Frontend initiates Google OAuth flow
3. User authenticates and consents on Google
4. Google returns IdToken to frontend
5. Frontend sends IdToken to `POST /auth/google`
6. Identity Service validates IdToken with Google's public keys
7. System checks if user exists (by email from Google claims)
8. If user exists → login; if not → create account with `Active` status
9. System generates and returns JWT

Alternative Flows:

- **A1 — Invalid IdToken:** System returns 401 at step 6
- **A2 — Google API unavailable:** System returns 503 with fallback to email login

Business Rules:

- SSO users have null `PasswordHash` (passwordless)
- No email verification needed for SSO users (Google-verified)
- SSO user account created with default `User` role

UC-04: Manage Profile

Description: An authenticated user views and updates their profile information.

Primary Actor: End User

Preconditions:

- User is authenticated with valid JWT

Postconditions:

- Profile changes saved and reflected immediately

Primary Flow:

1. User navigates to profile page
2. System fetches and displays current profile
3. User updates allowed fields (full name)

4. System validates input
5. System saves changes
6. System returns updated profile

Alternative Flows:

- **A1 — JWT expired:** System returns 401; frontend redirects to login
- **A2 — Invalid input:** System returns validation error at step 4

Business Rules:

- Email cannot be changed after registration
- Only `fullName` is editable by the user
- Password change requires separate flow (UC-12)

UC-05: Add Credit Card

Description: A user links a credit card to their account for billing and payments.

Primary Actor: End User

Preconditions:

- User is authenticated
- Card issuer exists in the system

Postconditions:

- Card stored with encrypted number
- Confirmation email sent

Primary Flow:

1. User navigates to card management page
2. User enters card number, cardholder name, expiry, issuer, credit limit, billing cycle day
3. System validates input (Luhn check, expiry date, issuer exists)
4. System encrypts card number (AES)
5. System generates masked number (`**** **** **** XXXX`)
6. System creates `CreditCard` record
7. System publishes `ICardAdded` event
8. Notification Service sends confirmation email
9. System returns card details (masked)

Alternative Flows:

- **A1 — Invalid card number:** System returns validation error at step 3

- **A2 — Card expired:** System returns validation error at step 3
- **A3 — Duplicate card:** System returns conflict error at step 3
- **A4 — Issuer not found:** System returns 404 at step 3

Business Rules:

- Card numbers are encrypted before storage
- CVV is never stored
- Soft delete only (no physical deletion)
- One card can be set as default

UC-06: Manage Cards

Description: A user views, updates, and soft-deletes their linked cards.
Primary Actor: End User
Preconditions:

- User is authenticated
- User has at least one linked card

Postconditions:

- Card changes applied and reflected

Primary Flow:

1. User navigates to card management page
2. System displays all active cards (excludes soft-deleted)
3. User selects a card to view/edit/delete
4. User performs action (update details, set default, or delete)
5. System validates and applies change
6. System returns updated card state

Alternative Flows:

- **A1 — Card not owned by user:** System returns 403
- **A2 — Card already deleted:** System returns 404
- **A3 — Setting default:** System unsets previous default card automatically

Business Rules:

- Deleted cards (`IsDeleted = true`) are excluded from all queries
- Setting a card as default unsets the previous default
- Card transactions cannot be deleted

UC-07: View Bills

Description: A user views their outstanding and historical bills.

Primary Actor: End User

Preconditions:

- User is authenticated
- User has at least one linked card

Postconditions:

- User can review current and past bills

Primary Flow:

1. User navigates to bills page
2. System fetches bills for user's cards
3. System displays bills grouped by status (Pending, Paid, Overdue, PartiallyPaid)
4. User selects a bill to view details
5. System displays bill breakdown (amount, min due, due date, card info)

Alternative Flows:

- **A1 — No bills exist:** System shows empty state message
- **A2 — Bill not owned by user:** System returns 403

Business Rules:

- Bills are generated per card per billing cycle
- Overdue bills are automatically marked by admin/system job
- Minimum due is a percentage of total bill amount

UC-08: View Statements

Description: A user views detailed billing statements with transaction breakdowns.

Primary Actor: End User

Preconditions:

- User is authenticated
- Statements exist for user's cards

Postconditions:

- User can review statement details and transaction history

Primary Flow:

1. User navigates to statements page
2. System fetches statements for user's cards
3. System displays statements with period summary
4. User selects a statement to view details
5. System displays full breakdown: opening balance, purchases, payments, refunds, charges, closing balance
6. User can view individual transactions within the statement

Alternative Flows:

- **A1 — No statements exist:** System shows empty state message

Business Rules:

- Statements are generated alongside bills
- Statement transactions are snapshots of card transactions in the billing period
- Statement data is immutable once generated

UC-09: Pay Bill with OTP

Description: A user pays an outstanding bill using wallet balance, confirmed with OTP verification.
Primary Actor: End User
Secondary Actors: Notification Service, Gmail SMTP, System (Saga)
Preconditions:

- User is authenticated
- Bill exists and is `Pending` or `PartiallyPaid`
- User has sufficient wallet balance (if wallet payment)

Postconditions:

- Payment completed via saga orchestration
- Bill status updated
- Wallet balance debited
- Card balance updated
- Rewards redeemed (if applicable)
- Confirmation email sent

Primary Flow:

1. User selects a bill and enters payment amount
2. User chooses payment type (Wallet or Card) and optionally enters rewards amount
3. System validates bill status, amount, and wallet balance

4. System creates `Payment` record (status: `Initiated`)
5. System generates 6-digit OTP (5-minute expiry)
6. System publishes `IPaymentOtpGenerated` event
7. Notification Service sends OTP email
8. System returns payment ID to user
9. User enters OTP from email
10. System validates OTP
11. System publishes `IStartPaymentOrchestration` event
12. Saga executes asynchronously:
    - Update bill status (Billing Service)
    - Redeem rewards if applicable (Billing Service)
    - Deduct card balance (Card Service)
13. Saga completes → payment marked `Paid`
14. System publishes `IPaymentCompleted`
15. Notification Service sends payment confirmation email

Alternative Flows:

- **A1 — Insufficient wallet balance:** System returns 422 at step 3
- **A2 — Bill already paid:** System returns 422 at step 3
- **A3 — OTP expired:** System returns 401 at step 10; user can resend (UC-09 resend)
- **A4 — OTP mismatch:** System returns 400 at step 10; user retries (max 3 attempts)
- **A5 — Saga step fails:** System triggers compensation (see UC-09 exception flow)

Exception Flow (Saga Compensation):

1. Saga detects failure at any step
2. System enters `Compensating` state
3. System reverses completed steps in reverse order:
    - Reverse reward redemption (if rewards were redeemed)
    - Revert bill status (if bill was updated)
    - Refund wallet balance (if wallet was debited)
4. Payment marked `Compensated`
5. System publishes `IPaymentCompensated`
6. Notification Service sends payment failure email

Business Rules:

- Payment OTP expires after 5 minutes
- Maximum 3 OTP verification attempts
- Rewards redemption is optional during payment
- Saga is idempotent — duplicate requests with same CorrelationId are ignored

- Expired payments are cleaned up by background job

UC-10: Redeem Rewards

Description: A user applies available reward points to reduce the payable amount during bill payment.
Primary Actor: End User
Preconditions:

- User is authenticated
- User has a reward account with positive points balance
- User is in the payment flow (UC-09)

Postconditions:

- Reward points deducted
- Payable amount reduced
- Reward transaction recorded

Primary Flow:

1. User initiates payment (UC-09, step 1-2)
2. System displays available reward points and monetary value
3. User enters rewards amount to redeem
4. System validates rewards amount (<= available balance)
5. System includes rewards amount in payment request
6. During saga execution, Billing Service deducts points
7. System creates `RewardTransaction` (type: `Redeemed`)

Alternative Flows:

- **A1 — No reward points:** System disables rewards input at step 2
- **A2 — Rewards exceed balance:** System returns validation error at step 4
- **A3 — Saga fails:** Rewards are automatically reversed (UC-09 exception flow)

Business Rules:

- Reward rate is determined by user's reward tier (network + issuer specific)
- Rewards earned on payment completion (separate from redemption)
- Redeemed rewards are reversible via saga compensation

UC-11: Top-Up Wallet

Description: A user adds funds to their wallet via Razorpay for future bill payments.

Primary Actor: End User

Secondary Actor: Razorpay

Preconditions:

- User is authenticated

Postconditions:

- Wallet balance increased
- Top-up transaction recorded

Primary Flow:

1. User navigates to wallet page
2. User enters top-up amount
3. System validates amount (> 0)
4. System creates Razorpay order
5. System returns `razorpayOrderId` to frontend
6. Frontend opens Razorpay Checkout
7. User completes payment on Razorpay
8. Razorpay sends webhook callback to Payment Service
9. System verifies Razorpay signature (HMAC-SHA256)
10. System updates wallet balance
11. System creates `WalletTransaction` (type: `TopUp`)
12. System marks `RazorpayWalletTopUp` as `Completed`

Alternative Flows:

- **A1 — Razorpay payment failed:** System marks top-up as `Failed` at step 8
- **A2 — Signature mismatch:** System rejects webhook at step 9
- **A3 — Duplicate webhook:** System ignores (idempotent handling) at step 8

Business Rules:

- Wallet is auto-created on first use
- Wallet balance cannot go negative
- Razorpay webhook signature must match for security
- Wallet transactions are immutable

UC-12: Reset Password

Description: A user resets their password using an OTP sent to their registered email.

Primary Actor: End User

Preconditions:

- User account exists and is verified
- User has access to registered email

Postconditions:

- Password updated
- Old sessions invalidated

Primary Flow:

1. User navigates to forgot password page
2. User enters email address
3. System validates email exists
4. System generates 6-digit password reset OTP (10-minute expiry)
5. System publishes `IUserOtpGenerated` event
6. Notification Service sends reset email with OTP
7. System returns success response
8. User enters email, OTP, and new password
9. System validates OTP and new password strength
10. System updates password hash
11. System returns success response

Alternative Flows:

- **A1 — Email not found:** System returns generic success (prevents user enumeration)
- **A2 — OTP expired:** System returns error at step 9; user requests new OTP
- **A3 — Weak new password:** System returns validation error at step 9

Business Rules:

- Reset OTP expires after 10 minutes
- New password must meet the same strength requirements as registration
- Password reset invalidates existing JWT tokens

UC-13: Manage Users

Description: An admin views, updates status, and changes roles for platform users.

Primary Actor: Admin

Preconditions:

- Admin is authenticated with `Admin` role

Postconditions:

- User status or role updated
- Audit log entry created

Primary Flow:

1. Admin navigates to user management page
2. System displays paginated list of all users
3. Admin selects a user to view details
4. Admin performs action (change status, change role, or view stats)
5. System validates request
6. System applies change
7. System creates `AuditLog` entry
8. System returns updated user state

Alternative Flows:

- **A1 — User not found:** System returns 404 at step 3
- **A2 — Insufficient privileges:** System returns 403 at step 4

Business Rules:

- Admins can change user status between `Active` and `Suspended`
- Suspended users cannot login
- Admins can promote users to `Admin` role or demote to `User`
- All admin actions are audit-logged with JSON change diffs

UC-14: Manage Reward Tiers

Description: An admin configures reward calculation rules (rates, thresholds, network/issuer targeting).

Primary Actor: Admin

Preconditions:

- Admin is authenticated with `Admin` role

Postconditions:

- Reward tier created, updated, or deleted
- New tier applies to future reward calculations

Primary Flow:

1. Admin navigates to reward tier management page
2. System displays existing reward tiers
3. Admin creates, updates, or deletes a tier
4. System validates input (network, rate, min spend, effective dates)
5. System saves changes
6. System returns updated tier list

Alternative Flows:

- **A1 — Invalid tier configuration:** System returns validation error at step 4
- **A2 — Duplicate tier:** System returns conflict error at step 4

Business Rules:

- Reward tiers target specific card networks and optionally specific issuers
- `issuerId = null` means the tier applies to all issuers of that network
- Tiers have effective date ranges
- Reward rate = points earned per unit of spend

UC-15: View Audit Logs

Description: An admin views the system-wide audit trail of administrative actions.

Primary Actor: Admin

Preconditions:

- Admin is authenticated with `Admin` role

Postconditions:

- Admin can review audit history

Primary Flow:

1. Admin navigates to audit logs page
2. System displays paginated audit log entries
3. Admin filters by entity, action, user, or date range

4. System displays filtered results with JSON change diffs

Alternative Flows:

- **A1 — No logs match filter:** System shows empty state

Business Rules:

- Audit logs are immutable
- Logs include: entity name, entity ID, action, acting user, JSON diff of changes, trace ID
- All admin actions (user status changes, role changes, bill generation) are audit-logged

5. Cross-Cutting Business Rules

Rule	Applies To	Description
Idempotency	All financial operations	Duplicate requests must not cause duplicate processing
Data Encryption	Card numbers	AES encryption at rest; masked display in UI
Audit Logging	Admin actions, payments, wallet updates	All significant changes logged with JSON diffs
OTP Validation	Registration, password reset, payments	6-digit, time-limited, single-use
Saga Compensation	Distributed payments	Failed steps trigger reverse actions for completed steps
Soft Deletes	Cards	`IsDeleted` flag instead of physical deletion
Token Expiry	JWT	60 minutes; auto-logout on 401
Rate Limiting	OTP, login	Prevents brute force and abuse

End of Use Case Specification